

SauceDemo QA Assessment Notes

Submission Information

****Public automation repository:****

<https://github.com/CliffBooth07/saucedemo-sqa-assessment>

This document is the single assessment report. The repository contains the runnable Selenium test, setup instructions, source files, and evidence artifacts referenced below. Evidence files are also available in the repository under `artifacts/`.

****Automation command:**** `pytest -q`

****Latest validation:**** `1 passed`

Tools and test basis

Testing was performed against `https://www.saucedemo.com/` with the `standard\_user` account. Tools used: Chrome, Selenium WebDriver, Python, pytest, browser DevTools, and GitHub Copilot for idea generation and code review. The automation uses Selenium 4's Selenium Manager, so a manually downloaded driver is not required.

Task A: AI-assisted exploratory testing

Prompt 1: Risk-based journey selection

****Prompt:**** For a retail demo app, rank the highest-risk user journeys by business impact and failure likelihood.

****Purpose:**** Focus the short exploratory session on the most important business flows.

****Useful output and validation:**** Login, cart state, checkout, and order confirmation were selected. Payment and account-recovery ideas were reduced because those features are not present in this application.

Prompt 2: Checkout exploratory questions

****Prompt:**** Generate exploratory questions for SauceDemo checkout, including validation and interrupted-session cases. Do not assume a payment service exists.

****Purpose:**** Find negative paths without inventing unsupported product behavior.

****Useful output and validation:**** Empty required fields, browser Back, refresh, and direct navigation were useful checks. Payment-specific ideas were rejected as out of scope.

Prompt 3: Locator maintainability

****Prompt:**** Review these Selenium locator candidates and rank them by maintenance risk: generated classes, visible text, IDs, and data-test attributes.

****Purpose:**** Choose selectors that are less brittle and easier to maintain.

****Useful output and validation:**** `id` and `data-test` selectors were preferred. Presentation-oriented CSS classes were avoided.

Prompt 4: Completion assertions

****Prompt:**** What assertions distinguish a completed order from merely reaching the checkout page?

****Purpose:**** Prevent a false-positive automation result.

****Useful output and validation:**** The test asserts cart count, product name, and the final `Thank you for your order!` confirmation. URL-only validation was rejected.

The AI suggestions were treated as hypotheses. I manually checked that the suggested selectors exist in the DOM and that the final confirmation is visible only after clicking Finish. Fixed sleeps and broad assertions were rejected because they make the test slower and less diagnostic.

Task B: defect discovery and reporting

Finding status

Five significant defects were reproduced manually in Chrome using the `standard\_user` account. SauceDemo is a demo application and does not process a real payment; references to payment below mean that the application allowed the order-completion flow to finish.

DEF-001: Checkout accepts invalid postal-code and personal-data characters

****Steps to reproduce:****

1. Login with `standard\_user` / `secret\_sauce`.
2. Add Sauce Labs Backpack and open the cart.
3. Click Checkout.
4. Enter `123` as First Name, `456` as Last Name, and `qwe` as Postal Code.
5. Click Continue.
6. Repeat with `Test`, `User`, and `abc`, and with `@#\$`, `%^^`, and `\*()`.
7. Click Finish on the summary page.

****Expected result:****

The form should reject values that do not match the application's required field formats, show a clear validation message, and prevent order completion until corrected.

****Actual result:****

Alphabetic, numeric, and special-character values were accepted. The application opened the summary page and allowed the order-completion flow to finish without showing a validation error.

****Severity:**** Medium

****Priority:**** High

****Business/user impact:****

Invalid customer and postal data can be stored against an order. In a real checkout system this could cause fulfilment, delivery, reporting, or customer-contact problems.

****Evidence:****

The checkout form with invalid values is shown in [DEF-001-invalid-checkout-data_1.png] (../artifacts/DEF-001-invalid-checkout-data_1.png), and the resulting overview page is shown in [DEF-001-invalid-checkout-data_2.png] (../artifacts/DEF-001-invalid-checkout-data_2.png).

DEF-002: Empty cart can proceed to checkout and complete an order

****Steps to reproduce:****

1. Login with `standard\_user` / `secret\_sauce`.
2. Add Sauce Labs Backpack and open the cart.
3. Click Remove.
4. Confirm that the cart item is removed and the cart is empty.
5. Click the still-available Continue to checkout button.
6. Enter any checkout values and complete the flow.

****Expected result:****

An empty cart should not allow checkout or order completion. The checkout action should be disabled or unavailable until at least one product is present.

****Actual result:****

After the only item was removed, the cart was empty but the checkout action remained usable. The flow proceeded to checkout and allowed order completion.

****Severity:**** High

****Priority:**** High

****Business/user impact:****

The application can create an order with no product lines. In a real commerce system this could create billing, inventory, fulfilment, and reconciliation errors.

****Evidence:****

The cart before removal is shown in [DEF-002-empty-cart-checkout_1.png] (../artifacts/DEF-002-empty-cart-checkout_1.png), the empty cart with Checkout still available is shown in [DEF-002-empty-cart-checkout_2.png] (../artifacts/DEF-002-empty-cart-checkout_2.png), and the resulting checkout page is shown in [DEF-002-empty-cart-checkout_3.png] (../artifacts/DEF-002-empty-cart-checkout_3.png).

DEF-003: Valid entered checkout fields are shown in an error state

****Steps to reproduce:****

1. Login with `standard\_user` / `secret\_sauce`.
2. Add Sauce Labs Backpack and open the cart.
3. Click Checkout.
4. Enter a value in First Name, for example `asd`.
5. Enter a value in Last Name, for example `qwe`.

6. Leave Zip/Postal Code empty.
7. Click Continue.

****Expected result:****

Only the missing Postal Code field should be marked invalid, with the corresponding validation message. Fields containing entered values should remain in their normal valid state.

****Actual result:****

The message correctly says `Error: Postal Code is required`, but the already-filled First Name field is also displayed with red error styling and an error icon. This makes a valid field appear to be the cause of the failure.

****Severity:**** Medium

****Priority:**** Medium

****Business/user impact:****

Users may clear or re-enter valid information unnecessarily and may not understand which field actually needs correction. This creates confusion during checkout and increases abandonment risk.

****Evidence:****

The before-validation state is shown in [DEF-003-incorrect-field-error-state_1.png](../artifacts/DEF-003-incorrect-field-error-state_1.png). The resulting red styling and Postal Code error are shown in [DEF-003-incorrect-field-error-state_2.png](../artifacts/DEF-003-incorrect-field-error-state_2.png).

DEF-004: Direct URL access bypasses checkout-step sequencing

****Steps to reproduce:****

1. Login with `standard\_user` / `secret\_sauce`.
2. Do not complete the checkout form.
3. Enter `https://www.saucedemo.com/checkout-step-two.html` directly in the browser address bar.
4. Repeat with an empty cart if required to confirm the state is not dependent on a selected product.

****Expected result:****

The application should enforce the checkout sequence. A user who has not completed Step One should be redirected to the cart or checkout Step One. An empty cart should not show an order overview.

****Actual result:****

The user can open the Step Two overview directly without entering First Name, Last Name, or Postal Code. With an empty cart, the overview can display a `\$0.00` total.

****Severity:**** High

****Priority:**** High

****Business/user impact:****

Users can bypass mandatory checkout steps and reach an order-review state with incomplete customer information or no products. In a real commerce application this could lead to invalid orders and broken checkout controls.

****Evidence:****

The direct navigation and bypass sequence is recorded in [DEF-004-direct-checkout-step-bypass.mp4] (../artifacts/DEF-004-direct-checkout-step-bypass.mp4). The recording shows the URL and the Step Two overview without completing Step One.

DEF-005: Reset App State does not reset product buttons immediately

****Steps to reproduce:****

1. Login with `standard\_user` / `secret\_sauce`.
2. Add two or three products to the cart.
3. Open the left burger menu.
4. Click `Reset App State`.
5. Close the menu and inspect the product cards and cart badge.

****Expected result:****

Reset App State should immediately reset all related UI state. Product buttons should change from `Remove` back to `Add to cart`, and the cart badge should disappear completely.

****Actual result:****

The cart data is removed, but one or more product buttons remain in the `Remove` state instead of reverting to `Add to cart`. The UI does not fully reflect the reset until a page refresh. Depending on browser/session state, an empty cart badge element may also remain in the DOM.

****Severity:**** Medium

****Priority:**** High

****Business/user impact:****

The user sees a stale cart state and may believe products are still selected. Clicking a stale `Remove` button can make product selection behavior confusing and reduces trust in the reset function.

****Evidence:****

The product state before opening the menu is shown in [DEF-005-reset-app-state-ui-desync_1.png] (../artifacts/DEF-005-reset-app-state-ui-desync_1.png), the Reset App State action is shown in [DEF-005-reset-app-state-ui-desync_2.png] (../artifacts/DEF-005-reset-app-state-ui-desync_2.png), and the stale `Remove` buttons after reset are shown in [DEF-005-reset-app-state-ui-desync_3.png] (../artifacts/DEF-005-reset-app-state-ui-desync_3.png).

Follow-up exploratory checks

These are additional high-value checks to execute and capture with browser evidence:

Check 1: Empty required checkout fields

****Expected result:**** A field-level validation message identifies the missing field, and no order is created.

****Risk if it fails:**** The user may be unable to recover clearly, or an invalid order may be created.

Check 2: Invalid postal-code format

****Expected result:**** The input is rejected with a useful validation message if the product requires a specific postal-code format.

****Risk if it fails:**** Bad customer or delivery data may be accepted.

Check 3: Refresh or Back during checkout

****Expected result:**** Cart and entered data follow a consistent, documented behavior after refresh or browser Back.

****Risk if it fails:**** The user may lose order context or accidentally submit a duplicate order.

Check 4: Remove the only cart item

****Expected result:**** The cart becomes empty, the badge is removed, and checkout cannot create an order.

****Risk if it fails:**** The application may allow an invalid empty order or show misleading cart state.

All five findings above are confirmed from manual reproduction with the attached evidence. The remaining checks are still test targets, not defects, until their actual behavior and evidence are captured.

Task C: automation review

What AI assisted

AI helped draft the page-object boundaries, identify the main business flow, suggest explicit Selenium waits, and review assertion strength. The final selectors, fixture behavior, and assertions were manually checked against the application DOM.

Corrections and limitations

- Generated examples used fixed delays; they were replaced with `WebDriverWait` conditions.
- URL-only assertions were replaced with visible state and confirmation assertions.
- The test uses one product and one account, so it does not cover sorting, multiple quantities, locked-out users, or session isolation across browsers.
- The test depends on the public demo site and network availability. It should not be the only CI quality gate.

Task D: AI-driven QA proposal

AI can accelerate risk brainstorming, test-data generation, locator review, failure clustering, coverage-gap analysis, release-note checks, and draft defect reports. QA owns the final risk assessment: every generated test must be reviewed for business intent, independence, deterministic setup, meaningful assertions, security/privacy concerns, and false-positive risk before entering regression.

Human review is mandatory for acceptance criteria, severity and priority, production-impact decisions, security and privacy findings, accessibility conclusions, and any result based on incomplete or ambiguous evidence. AI-generated tests should run once against a known baseline, be reviewed by a code owner, and prove that they fail for the intended defect before being trusted.

In CI/CD, a small smoke suite can gate merges, broader functional regression can gate release candidates, and flaky-test trends should be visible rather than hidden by automatic retries. AI may group failures and suggest likely ownership, but it must link each conclusion to logs, screenshots, traces, or reproducible steps. Coverage analysis should combine requirement coverage, risk coverage, and production telemetry; line coverage alone is insufficient.

Main risks are hallucinated requirements, brittle selectors, duplicated tests, masked failures, insecure test data, and overconfidence in generated output. Mitigations are source-grounded prompts, human approval, isolated test data, deterministic waits, bounded retries, secret scanning, audit logs for AI changes, and regular review of the regression suite's value and flake rate.